

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 1, Section 1.1
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

BASICS OF FINANCIAL DERIVATIVES

Derivative Pricing - Module 1



Outline

- ▶ What is a financial derivative?
- ▶ Revisiting the main features of option contracts
- ▶ Intro to binomial model for equity option pricing

Financial Derivatives: What are they?

During the Financial Markets course, you got familiar with financial derivatives and learned about their powerful applications in the real world for players on the **buy-side** (e.g., hedge funds) and the **sell-side** (e.g., investment banks) of the industry.

- Investors find these instruments convenient for a variety of reasons (e.g., hedging, leverage) → Volume traded on **equity option market**:

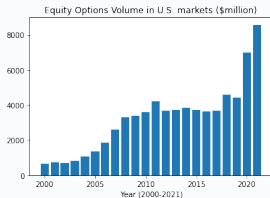


Figure: Constructed with data from Options Clearing Corporation

In this course, you'll learn the basics of pricing some of these instruments.

Financial Derivatives: What are they?

Given their importance, we will focus on **equity options** during most of the course. Some important features of these contracts that we need to know:

- ▶ Call vs. Put
- ▶ Moneyness & Liquidity → OTM/ATM/ITM?
- ▶ European vs. American → early exercise?
- ▶ Non-linear payoffs
- ▶ Leverage → e.g., 100 shares of underlying stock
- ▶ Over-the-Counter (OTC) vs. Exchange-Traded → credit risk?
- ▶ Exposure to volatility → e.g., Butterfly / Straddles

An **option contract** is a contingent claim on an underlying asset with a pre-defined payoff:

- ▶ Call Option: $(S_t - K)^+ = \max(S_t - K; 0)$,
where S_t is underlying stock price at time t and K is the option's strike price.
- ▶ Put Option: $(K - S_t)^+ = \max(K - S_t; 0)$

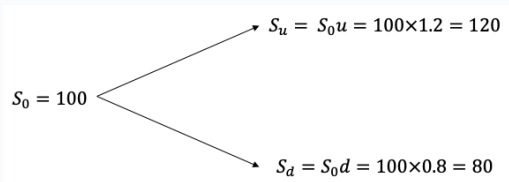
Modeling Underlying Stock

The essential characteristic of a derivative is that its payoff depends on the evolution of an **underlying asset** (e.g., a stock for equity options).

- Thus, a key aspect for pricing (and our first step) is to model the future evolution of the underlying asset.

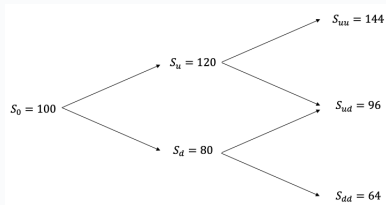
Probably one of the simplest and more popular techniques for modeling underlying stock price evolution is the **binomial tree** ([Cox, Ross, and Rubinstein, 1979](#)):

- A binomial model assumes that stock prices can make, at any point in time, upward (u) or downward (d) movements. For example, for $u = 1.2$ and $d = 0.8$:



Binomial Tree

We can expand this tree to recognize any amount of steps (e.g., $N = 2$)...

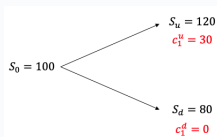


What is the information known at each node? → concept of **filtration**!

- ▶ For example, if we are on $S_{ud} = 96$, we know that the stock price climbed on the first step and descended on the second. For $S_t = 96$, then, filtration is $\{u, d\}$.
- ▶ What is the filtration of $S_t = 64$?
- ▶ This will be an important concept moving forward especially when we get to probability theory.

Binomial Tree & Call Option

Once we have an underlying stock price evolution, getting the payoff of any simple option is very easy. Picture a European Call Option with strike price $K = 90$, maturity $T = 1$ year, and a 1-step ($N = 1$) binomial tree:



Given payoffs, basic financial intuition would tell us that the **fair value** of such an option is just the present value of expected payoff, which raises two important questions:

- For *expected payoff*, what are the real-world probabilities of u and d happening?
- For *present value*, what is the discount rate that we should use?

The answers to both of these questions rest on the concept of **risk-neutral valuation**.

No-Arbitrage Argument

Let's ask ourselves the following: Can we construct a portfolio that yields a riskless payoff?

- ▶ Consider a portfolio with X amount of shares of the underlying and a short position in the call option (i.e., seller of the option).
- ▶ Call payoff can move by \$30 ($c_1^u - c_1^d$) while underlying price moves by \$40 ($S_u - S_d$).
- ▶ Thus, we need to hold 0.75 underlying shares ($= 30/40$) to be exposed to future movements like the call option.

What will be the value of such a portfolio in the future ($T = 1$)?

- ▶ If the stock climbs to \$120, we have a 'u' scenario:

$$\$120 \times 0.75 - \$30 = \$60$$

- ▶ If the stock drops to \$80, we have a 'd' scenario:

$$\$80 \times 0.75 - \$0 = \$60$$

No-Arbitrage & Option Pricing

Investors can build a portfolio that yields a value with certainty making it riskless.

- ▶ That \$60 is the value of the portfolio one year from now.
- ▶ Since it is a riskless portfolio, we can discount at risk-free rate $\rightarrow$ Suppose $r_f = 10\%$ a year:

$$\text{Portfolio Value Today} = 60 \times e^{-0.1 \times 1} = \$54.29$$

- ▶ **Under no-arbitrage**, if underlying today is \$100, Call today (C_0) will be worth...

$$\text{Portfolio Value Today} = \$100 \times 0.75 - C_0 = \$54.29 \rightarrow C_0 = \$20.71$$

This exercise provides **two important results** for option pricing:

- ▶ We can price an option as part of a riskless portfolio $\rightarrow$ Risk-free discount rate
- ▶ Under these conditions, we can obtain implied probabilities (risk-neutral).

The latter two concepts are the foundation of risk-neutral valuations.

$\rightarrow$ We can think of investors as risk-neutral!

Intro to Risk-Neutral Probabilities

Absent arbitrage, we get the value of riskless portfolio & call option → Probabilities?

- Present value of our riskless portfolio is the portfolio cost today. For u :

$$S_0X - C_0 = (S_0uX - c_1^u)e^{-rT} \rightarrow C_0 = S_0X(1 - ue^{-rT}) + c_1^ue^{-rT}$$

- We know that $X = \frac{c_1^u - c_1^d}{S_u - S_d}$, so substituting and rearranging:

$$C_0 = S_0 \frac{c_1^u - c_1^d}{S_u - S_d} (1 - ue^{-rT}) + c_1^ue^{-rT} \rightarrow C_0 = \frac{c_1^u(1 - de^{-rT}) + c_1^d(ue^{-rT} - 1)}{u - d} \rightarrow$$

which can be written as:

$$C_0 = e^{-rT} [pc_1^u + (1 - p)c_1^d], \text{ where } p = \frac{e^{rT} - d}{u - d}$$

- That, in our previous example with $u = 1.2$, $d = 0.8$, and $r_f = 10\%$ would be:

$$p = \frac{e^{0.1 \times 1} - 0.8}{1.2 - 0.8} = 0.7629$$

This (p) is our risk-neutral probability measure.

Summary of Lesson 1

In this intense Lesson 1, we have covered several important concepts:

- ▶ Basic features of financial derivatives (options in particular)
- ▶ Binomial model to simulate the behavior of the underlying asset
- ▶ Option payoffs and riskless portfolio in binomial tree
- ▶ No-arbitrage argument to find option price
- ▶ Intro to risk-neutral valuation

⇒ **TO-DO NEXT:** In the Jupyter Notebook accompanying this lesson, you will find an example of how to build a function to construct a binomial tree in Python. Here, you will also see how, using backward induction, we can get the price of the call option.

⇒ In the next lesson, we will go deeper into the replication and risk-neutral valuation ideas by considering a put option and its relationship to the call option case.

Lesson 1 - Financial Derivatives Basics

February 6, 2026

0.1 DERIVATIVE PRICING

MODULE 1 | LESSON 1

1 FINANCIAL DERIVATIVE BASICS

Reading Time	75 minutes
Prior Knowledge	Options, financial derivatives basics
Keywords	Option, Call, Put, Risk-neutral probabilities, Real-world probabilities, No-arbitrage argument, Binomial model

—All rights reserved WQU WorldQuant University QQQQ

Here you have the first Jupyter Notebook of your Derivative Pricing course. We will use this format throughout the whole course in order to do a proper mapping of the theoretical concepts in the slides/videos to the hands-on development of the ideas.

In this notebook, we will develop a function in Python in order to construct a simple binomial tree. Make sure you understand this properly because it is going to set the foundation for future work.

```
[1]: import os
import sys

# Get the full path of the current notebook
notebook_path = os.path.join(os.getcwd(), sys.argv[0])
print(notebook_path)
```

/usr/local/lib/python3.11/site-packages/ipykernel_launcher.py

Let's start importing some of the libraries that we may need to use down the road:

```
[2]: import numpy as np
```

In order to compute the binomial tree in the video/slides of Lesson 1, we need a bunch of input data: upward movement (**u**), downward movement (**d**), risk-free rate (r_f), time-horizon (**T**), and number of steps in the tree (**N**).

All these would be user-inputs to our function. However, there is one input that we need to consider with caution: the **time-step**. That is, how long does moving from one node to the next one in the tree represent in terms of time?

In our baseline example on the slides the time-horizon for the evolution of the underlying (T) was equal to 1 year. There was 1 step in the tree ($N=1$). Therefore, there were no doubts that the time-step needed to be equal to 1.

What if we want a different number of steps, or the time-horizon for the evolution of the underlying is different? In those cases, the time-step (dt) we use for our binomial model will be $\rightarrow dt = T/N$.

Although we have not yet introduced the utility to specify dt , you will realize soon that dt will be helpful for the calculation of risk-neutral probabilities.

1.1 1. Constructing a Binomial Tree

In the following code snippet, you have the code for a general function to simulate the underlying stock price for some inputs: initial stock price (S_{ini}), time-horizon (T), upward (u) and downward (d) movements, and number of steps (N).

```
[3]: def binomial_tree(S_ini, T, u, d, N):
      S = np.zeros([N + 1, N + 1]) # Underlying price
      for i in range(0, N + 1):
          S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
      for j in range(N - 1, -1, -1):
          for i in range(0, j + 1):
              S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
      return S
```

Note that we store everything in a variable, S , the one returned by the function. This variable will contain an array with the values of the stock price at each point in time in a lower triangular matrix.

Let's check it by replicating the same tree for $N = 2$ that you have in the Lesson 1 slides:

```
[4]: Stock = binomial_tree(100, 1, 1.2, 0.8, 2)
      Stock
```

```
[4]: array([[100.,  0.,  0.],
           [ 80., 120.,  0.],
           [ 64.,  96., 144.]])
```

1.2 2. Extending the Tree with Call Option Payoffs

Next, let's extend the previous function by adding another variable that computes the payoffs associated with a Call Option of certain characteristics. Note that we are focusing on a European Call Option with strike price $K = 90$, and therefore the payoff is only computed at maturity:

```
[5]: def binomial_tree_call(S_ini, K, T, u, d, N):
      C = np.zeros([N + 1, N + 1]) # Call prices
      S = np.zeros([N + 1, N + 1]) # Underlying price
```

```

for i in range(0, N + 1):
    C[N, i] = max(S_ini * (u ** (i)) * (d ** (N - i)) - K, 0)
    S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
for j in range(N - 1, -1, -1):
    for i in range(0, j + 1):
        S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
return S, C

```

It is easy to see that the variable C output by the function will return Call Option payoff at maturity. We can verify this by replicating the simple $N=1$ tree with Call option payoff from the Lesson 1 slides:

```

[6]: Stock, Call = binomial_tree_call(100, 90, 10, 1.2, 0.8, 10)
print("Underlying Price Evolution:\n", Stock)
print("Call Option Payoff:\n", Call)

```

Underlying Price Evolution:

```

[[100.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 80.      120.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 64.      96.      144.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 51.2      76.8      115.2      172.8      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 40.96      61.44      92.16      138.24      207.36
   0.      0.      0.      0.      0.
   0.      ]
 [ 32.768      49.152      73.728      110.592      165.888
  248.832      0.      0.      0.      0.
   0.      ]
 [ 26.2144      39.3216      58.9824      88.4736      132.7104
  199.0656      298.5984      0.      0.      0.
   0.      ]
 [ 20.97152      31.45728      47.18592      70.77888      106.16832
  159.25248      238.87872      358.31808      0.      0.
   0.      ]
 [ 16.777216      25.165824      37.748736      56.623104      84.934656
  127.401984      191.102976      286.654464      429.981696      0.
   0.      ]
 [ 13.4217728      20.1326592      30.1989888      45.2984832      67.9477248
  101.9215872      152.8823808      229.3235712      343.9853568      515.9780352
   0.      ]
 [ 10.73741824      16.10612736      24.15919104      36.23878656      54.35817984

```


where dt is the discounted period from one node to the next (**time-step**), and c_{t+1}^u and c_{t+1}^d are the **values of the Call option in the next period**. We will therefore have to start from the last period (maturity) and work backwards, hence the term backward induction.

Note that we use dt here because we are assuming there are a **bunch of periods (steps) in the tree from the initial date until maturity of the option**. Under the 1-step case, we can calculate risk-neutral probabilities as we did in the videos, because $dt = T/N = 1/1 = 1 = T$:

$$p = \frac{e^{rT} - d}{u - d}$$

Once we consider a different dt , we just need to modify p accordingly:

$$p = \frac{e^{r dt} - d}{u - d}$$

Now, let's write a final function that recognizes all these issues.

```
[7]: def binomial_call_full(S_ini, K, T, r, u, d, N):
    dt = T / N # Define time step
    p = (np.exp(r * dt) - d) / (u - d) # Risk neutral probabilities (probs)
    C = np.zeros([N + 1, N + 1]) # Call prices
    S = np.zeros([N + 1, N + 1]) # Underlying price
    for i in range(0, N + 1):
        C[N, i] = max(S_ini * (u ** (i)) * (d ** (N - i)) - K, 0)
        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            C[j, i] = np.exp(-r * dt) * (p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i])
            S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
    return C[0, 0], C, S
```

Notice that since we are doing backward induction, the first value of the Call Option Payoff matrix (the last we calculate) is the price of the Call Option today.

Let's replicate with the values from the example in the slides to check it:

```
[8]: call_price, C, S = binomial_call_full(100, 90, 10, 0, 1.2, 0.8, 10)
print("Underlying Price Evolution:\n", S)
print("Call Option Payoff:\n", C)
print("Call Option Price at t=0: ", "{:.2f}".format(call_price))
```

Underlying Price Evolution:

```
[[100.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 80.     120.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 64.      96.     144.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 51.2     76.8    115.2    172.8     0.
   0.      0.      0.      0.      0.
   0.      ]]
```

0.	0.	0.	0.	0.
0.	]			
[40.96	61.44	92.16	138.24	207.36
0.	0.	0.	0.	0.
0.	]			
[32.768	49.152	73.728	110.592	165.888
248.832	0.	0.	0.	0.
0.	]			
[26.2144	39.3216	58.9824	88.4736	132.7104
199.0656	298.5984	0.	0.	0.
0.	]			
[20.97152	31.45728	47.18592	70.77888	106.16832
159.25248	238.87872	358.31808	0.	0.
0.	]			
[16.777216	25.165824	37.748736	56.623104	84.934656
127.401984	191.102976	286.654464	429.981696	0.
0.	]			
[13.4217728	20.1326592	30.1989888	45.2984832	67.9477248
101.9215872	152.8823808	229.3235712	343.9853568	515.9780352
0.	]			
[10.73741824	16.10612736	24.15919104	36.23878656	54.35817984
81.53726976	122.30590464	183.45885696	275.18828544	412.78242816
619.17364224]				

Call Option Payoff:

```
[2.93845445e+01 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.57572106e+01 4.30118784e+01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[7.17743934e+00 2.43369819e+01 6.16867749e+01 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[2.49687648e+00 1.18580022e+01 3.68159616e+01 8.65575882e+01
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[5.04779760e-01 4.48897320e+00 1.92270312e+01 5.44048920e+01
1.18710284e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 1.00955952e+00 7.96838688e+00 3.04856755e+01
7.83241085e+01 1.59096460e+02 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 2.01911904e+00 1.39176547e+01
4.70536963e+01 1.09594521e+02 2.08598400e+02 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 4.03823808e+00
2.37970714e+01 7.03103213e+01 1.48878720e+02 2.68318080e+02
0.00000000e+00 0.00000000e+00 0.00000000e+00]
```

```

[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 8.07647616e+00 3.95176666e+01 1.01102976e+02 1.96654464e+02
 3.39981696e+02 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 1.61529523e+01 6.28823808e+01 1.39323571e+02
 2.53985357e+02 4.25978035e+02 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 3.23059046e+01 9.34588570e+01
 1.85188285e+02 3.22782428e+02 5.29173642e+02]]
Call Option Price at t=0: 29.38

```

Feel free to play around with these functions to get familiar with how they work.

1.4 4. Conclusion

In this lesson, we have started to price options in the simple framework of the binomial model. In the next lesson, we will keep working with the binomial model on features like the put-call parity.

See you in the next lesson!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 1, Section 1.2
- **Estimated reading time:** 25 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 2

OPTION PRICING AND PUT-CALL PARITY

Derivative Pricing - Module 1



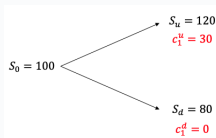
Outline

- ▶ Replicating Portfolio for Call and Put Options
- ▶ Pricing Put Options
- ▶ Put-Call Parity

Generalizing: Replicating portfolios

⇒ An option (call or put) can be replicated with a self-financed portfolio combining:

1. A position (long or short) in shares
2. A position in a risk-free bond (lend or borrow)



With X number of shares and B amount in the risk-free bond, such that (assume $r=0$ for simplicity):

$$XS_u + B = c_1^u \rightarrow X120 + B = 30$$

$$XS_d + B = c_1^d \rightarrow X80 + B = 0 \Rightarrow \text{Solving these: } X = 0.75, B = -60$$

These numbers are no coincidence to what we previously used.

- ▶ At $t=0$ we would borrow \$60 ($Be^{-rT}, r=0$), and invest \$75 dollars in the stock ($= 0.75 \times S_0$).
- ▶ This would yield exactly the same payoff as the call option.
- ▶ Part of the portfolio is self-financed (\$'s borrowed at r). The \$15 not financed is, precisely, the price of the call option today.

Replicating Portfolios for Put Option

We can do the same thing for a European put option with the same strike, $K = 90$:



► Check that you know how to obtain the replicating portfolio.

It is easy to check that, for the put case, $X = -0.25$ and $B = 30$.

→ Likewise, the difference in the self-financed portfolio is the value of the option at $t=0$:

$$-0.25S_0 + 30e^{-rT} = p_0 \rightarrow p_0 = -0.25 \times 100 + 30 = \$5$$

→ The same value as via backward induction with risk-neutral probabilities:

$$p_0 = e^{-rT}[\mathbf{p}p_1^u + (1 - \mathbf{p})p_1^d]$$

Please, do not mix up $\mathbf{p}$ (risk-neutral prob) with p_t (value of the put option)!

Put-Call Parity

Let's now explore the relationship between the value of the call and put options.

- To do so, we are going to build on the previous replicating portfolios:

$$\begin{aligned}0.75S_0 - 60e^{-rT} &= c_0 \\ -0.25S_0 + 30e^{-rT} &= p_0\end{aligned}$$

which we can put as:

$$0.75S_0 - 60e^{-rT} - c_0 = -0.25S_0 + 30e^{-rT} - p_0$$

and rearranging:

$$c_0 + 90e^{-rT} = S_0 + p_0$$

but the 90 is no coincidence; it is the strike of both options, K :

$$c_0 + Ke^{-rT} = S_0 + p_0$$

The latter expression is known as put-call parity.

Summary of Lesson 2

In this lesson, we have learned:

- ▶ Constructed replicating portfolios for call & put.
- ▶ Use of a binomial model to simulate the behavior of the underlying asset.
- ▶ Option payoffs and riskless portfolios in a binomial tree.
- ▶ Use of the no-arbitrage argument to find option price.
- ▶ Risk-neutral valuation.

⇒ **TO-DO NEXT:** In the Jupyter Notebook accompanying this lesson, you will find an example extending the binomial tree from Lesson 1. We will use the extended tree to do a more complex pricing of options and verify whether put-call parity holds.

⇒ In the next lesson, we will continue building on the concepts of replication and risk exposure of an investor with a position in an option. Moreover, can investors hedge their risk exposure?

Lesson 2 - Option Pricing and Put-Call Parity

February 6, 2026

0.1 DERIVATIVE PRICING

MODULE 1 | LESSON 2

1 OPTION PRICING AND PUT-CALL PARITY

Reading Time	40 minutes
Prior Knowledge	Financial Derivatives basics, Binomial tree, Risk neutral probabilities
Keywords	Risk neutral probabilities, Binomial model, No-arbitrage, Put-Call parity

In this notebook, we will continue to develop the binomial model for option pricing in Python. Building on the work we did in Lesson 1, we will now focus on relevant features like the put-call parity. Make sure you understand this properly because it is going to lay the foundation for future work.

As usual, let's start importing some of the libraries that we may need to use down the road:

```
[1]: import numpy as np
```

Our first task is going to be to extend the binomial model function used in Lesson 1 to also consider put options:

1.1 1. Binomial Tree for Put Options

We will create another function, similar to what we did with the call option case, to price put options. The following snippet contains the code for the call option we used in Lesson 1:

```
[2]: def binomial_call_full(S_ini, K, T, r, u, d, N):  
    dt = T / N # Define time step  
    p = (np.exp(r * dt) - d) / (u - d) # Risk neutral probs  
    C = np.zeros([N + 1, N + 1]) # Call prices  
    S = np.zeros([N + 1, N + 1]) # Underlying price  
    for i in range(0, N + 1):  
        C[N, i] = max(S_ini * (u ** (i)) * (d ** (N - i)) - K, 0)
```

```

        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            C[j, i] = np.exp(-r * dt) * (p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i])
            S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
    return C[0, 0], C, S

```

Before continuing, can you think of how to modify this accordingly to construct another function that prices put options?

```

[3]: def binomial_put_full(S_ini, K, T, r, u, d, N):
    dt = T / N # Define time step
    p = (np.exp(r * dt) - d) / (u - d) # Risk neutral probs
    P = np.zeros([N + 1, N + 1]) # Call prices
    S = np.zeros([N + 1, N + 1]) # Underlying price
    for i in range(0, N + 1):
        P[N, i] = max(K - (S_ini * (u ** (i)) * (d ** (N - i))), 0)
        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            P[j, i] = np.exp(-r * dt) * (p * P[j + 1, i + 1] + (1 - p) * P[j + 1, i])
            S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
    return P[0, 0], P, S

```

Let's verify this is correct with the numbers from the example in the slides and videos:

```

[4]: put_price, P, S = binomial_put_full(36, 36, 50, 0.01, 1.1, 0.7, 50)
print("Underlying Price Evolution:\n", S)
print("Put Option Payoff:\n", P)
print("Price at t=0 for Put option is$", "{:.2f}".format(put_price))

```

Underlying Price Evolution:

```

[[3.60000000e+01 0.00000000e+00 0.00000000e+00 ... 0.00000000e+00
 0.00000000e+00 0.00000000e+00]
 [2.52000000e+01 3.96000000e+01 0.00000000e+00 ... 0.00000000e+00
 0.00000000e+00 0.00000000e+00]
 [1.76400000e+01 2.77200000e+01 4.35600000e+01 ... 0.00000000e+00
 0.00000000e+00 0.00000000e+00]
 ...
 [1.32132126e-06 2.07636197e-06 3.26285453e-06 ... 3.49262042e+03
 0.00000000e+00 0.00000000e+00]
 [9.24924879e-07 1.45345338e-06 2.28399817e-06 ... 2.44483429e+03
 3.84188246e+03 0.00000000e+00]
 [6.47447415e-07 1.01741737e-06 1.59879872e-06 ... 1.71138400e+03
 2.68931772e+03 4.22607070e+03]]

```

Put Option Payoff:

```

[[ 7.5822903  0.          0.          ...  0.          0.

```

```

0.      ]
[ 9.54336876  7.11166527  0.      ...  0.      0.
0.      ]
[11.63290642  9.06090302  6.63837335 ...  0.      0.
0.      ]
...
[35.28715092 35.28715016 35.28714898 ...  0.      0.
0.      ]
[35.64179309 35.64179256 35.64179173 ...  0.      0.
0.      ]
[35.99999935 35.99999898 35.9999984  ...  0.      0.
0.      ]]

```

Price at t=0 for Put option is\$ 7.58

Next, we will verify whether put-call parity holds in this example at t=0.

1.2 2. Put-Call Parity in the Binomial Tree

We know that the price of the call and put options with K=90 at t=0 is given by the previous functions:

```

[5]: put_price, P, S = binomial_put_full(100, 90, 10, 0, 1.2, 0.8, 10)
print("Price at t=0 for Put option is $", "{:.2f}".format(put_price))
call_price, C, S = binomial_call_full(100, 90, 10, 0, 1.2, 0.8, 10)
print("Price at t=0 for Call option is $", "{:.2f}".format(call_price))

```

Price at t=0 for Put option is \$ 19.38

Price at t=0 for Call option is \$ 29.38

We already know that in order to satisfy absence of arbitrage conditions, the relationship between the price of the put and the call options must be:

$$C_0 + Ke^{-rT} = S_0 + P_0$$

Let's check whether this is the case for t=0:

```

[6]: round(call_price + 90 * np.exp(-0 * 1), 2) == round(S[0, 0] + put_price, 2)

```

```

[6]: np.True_

```

We have verified that the put-call parity here holds perfectly at t=0. The question that arises now is does this behavior also hold at other nodes of the tree (other times)?

In order to check this, let's first increase the number of steps of the binomial tree to N=2:

```

[7]: put_price, P, S = binomial_put_full(100, 90, 1, 0, 1.2, 0.8, 2)
print("Price at t=0 for Put option is $", "{:.2f}".format(put_price))
call_price, C, S = binomial_call_full(100, 90, 1, 0, 1.2, 0.8, 2)
print("Price at t=0 for Call option is $", "{:.2f}".format(call_price))

```

Price at t=0 for Put option is \$ 6.50

Price at t=0 for Call option is \$ 16.50

For completeness, let's start verifying Put-Call parity at $t=0$:

```
[8]: round(call_price + 90 * np.exp(-0 * 1), 2) == round(S[0, 0] + put_price, 2)
```

```
[8]: np.True_
```

Now, let's check the same thing for some other node of the two-step tree. Remember that this is the evolution of underlying prices and option payoffs:

```
[9]: print("Underlying Price Evolution:\n", S)
      print("Call Option Payoff:\n", C)
      print("Put Option Payoff:\n", P)
```

Underlying Price Evolution:

```
[[100.  0.  0.]
```

```
 [ 80. 120.  0.]
```

```
 [ 64.  96. 144.]]
```

Call Option Payoff:

```
[[16.5  0.  0.]
```

```
 [ 3.  30.  0.]
```

```
 [ 0.   6.  54.]]
```

Put Option Payoff:

```
[[ 6.5  0.  0.]
```

```
 [13.  0.  0.]
```

```
 [26.  0.  0.]]
```

Let's check put-call parity for the node following path 'd' (underlying price $S_d = 80$), which we can index as [1,0] in the matrix S:

```
[10]: round(C[1, 0] + 90 * np.exp(-0 * 0.5), 2) == round(S[1, 0] + P[1, 0], 2)
```

```
[10]: np.True_
```

Now, it is your turn to check whether this relationship holds for other nodes (you can also try trees with more steps) → Does put-call parity hold?

1.3 3. Conclusion

In this lesson, we have worked throughout the binomial tree to verify the put-call parity on certain nodes. In the next lesson, we will work on the concept of delta-hedging.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 1, Section 1.3
- **Estimated reading time:** 30 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

DELTA AND DELTA HEDGING

Derivative Pricing - Module 1



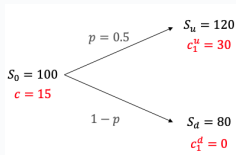
Outline

- ▶ Revisiting the delta concept formally
- ▶ Delta Hedging
- ▶ Putting it all together under the binomial framework

Delta: What We Already Know

In previous lessons, we showed how to obtain risk-neutral probabilities by constructing a replicating portfolio. We did this by calculating the ratio of the change in option price vs. the change in the price of the underlying asset. That is the concept behind **delta** (Δ).

Picture the following one-step binomial tree with $r = 0$ for a call option with $K = 90$:



Delta (Δ) measures the change in the price of the option with respect to the change in the price of the underlying asset.

$$\Delta_0 = \frac{c_1^u - c_1^d}{S_u - S_d} = \frac{30 - 0}{120 - 80} = 0.75$$

Delta: Extending the Tree

Things get more complicated as you extend the tree to contain more nodes, but the intuition on Δ remains the same.

Think of a tree similar to the one before but with $N=2$:

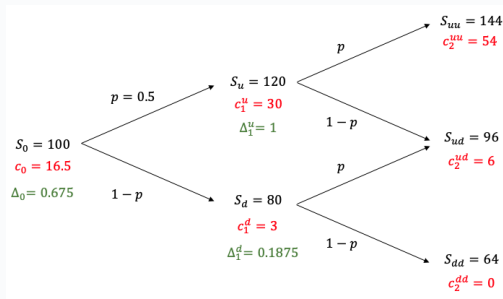


Figure: Binomial tree with stock prices (black), call option prices (red), and delta (green).

Delta and Delta Hedging

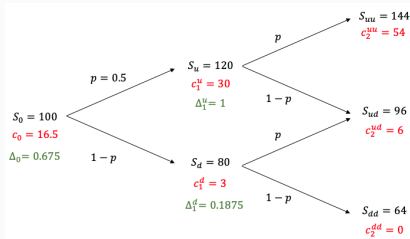
Delta is a classic example of a so-called *Greek* in derivatives. These Greeks are essentially variables that measure different sensitivities of option prices to several dimensions. We will dig deeper into these in upcoming modules.

For now, let's review a few features from delta to highlight:

- ▶ Δ measures option price sensitivity to changes in the price of the underlying.
- ▶ One important application is to construct *replicating portfolios* that are risk-free (remember previous lessons).
- ▶ The latter implicitly means that we can use Δ as a measure to **hedge** the exposure of our portfolio to underlying price changes → **Delta Hedging**

Let's revisit the example in the previous slide to understand the role of Δ in constructing a hedged position...

Delta Hedging: An Example



$\Rightarrow$ What would the hedge look like? (Assume that the underlying asset is a stock and that we are the bank selling this call)

- At $t = 0$, we need to hold 0.675 shares of the stock.
- At $t = 1$, we will need to hold 0.1875 shares if the underlying price goes down (S_d) or 1 share if it moves up (S_u).

Delta Hedging: An Example

- How would this **Delta-hedge** work? → Example for seller & path 'du':

	$t = 0$	$t = 1$	$t = 2$	Total
Underlying (stock) price	\$100	\$80	\$96	
Call option	\$16.5	\$3	\$6	
Δ hedge	0.675	0.1875		
Stock portfolio value	\$67.5	\$28.5	\$18	
Cash account	-\$67.5	+\$39	+\$18 - \$6	-\$16.5

At $t = 0$, we buy 0.657 shares of the underlying ($0.675 \times 100 = \$67.5$).

At $t = 1$, we sell 0.4875 shares ($= 0.675 - 0.1875$) to achieve the $\Delta = 0.1875$. For that, we obtain \$39 ($= 0.4875 \times 80$). The value of our stock portfolio at $t = 1$ is therefore \$28.5 ($= 67.5 - 39$) and we own 0.1875 shares.

At $t = 2$, our 0.1875 shares of the underlying will be worth \$18 ($= 0.1875 \times 96$). Since $t = 2$ is the maturity of the option contract, we get that \$18 from selling the stock. But the option buyer will come to collect its payoff, which we, as the seller, will have to pay (\$6).

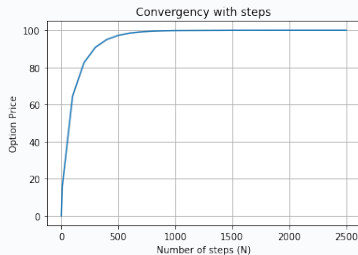
You can easily see that it is no coincidence that the total cost of the hedge is exactly the price of the call option at $t = 0$!

- But where is the profit for the Bank? → Fees!

Generalization to Any N

Now that we have understood the basics of pricing options through a binomial model and delta-hedging our position, let's check its power by increasing N .

The accompanying Jupyter Notebook contains a testing example of this based on the previous framework:



- ▶ Seems absurd that the call option price actually converges to S_0 ...Why?
- ▶ **Answer:** Careful with volatility! We need to adjust variability (i.e. u and d) to N .

Summary of Lesson 3

In this lesson, we have learned how to:

- ▶ Define and develop the concept of delta and delta hedging
- ▶ Calculate delta in the binomial model framework
- ▶ Use the binomial model for constructing delta-hedged portfolios
- ▶ Dynamically adjust the hedging in the binomial tree

⇒ **TO-DO NEXT:** In the Jupyter Notebook accompanying this lesson, you will find an example of how to construct delta hedging in the binomial tree in Python.

⇒ In the next lesson, we will go deeper into how to calibrate the binomial model to recognize volatility of the asset and revisit all the features covered in this module under a complete framework.

Lesson 3 - Introducing Delta

February 6, 2026

0.1 DERIVATIVE PRICING

MODULE 1 | LESSON 3

1 INTRODUCING DELTA

Reading Time	45 minutes
Prior Knowledge	Binomial tree for stock price evolution, Pricing options under a binomial tree, No-arbitrage argument
Keywords	Delta of a security, Delta hedging, Putting all the previous ideas together

*In this lesson, we are going to revisit and reinforce some previous concepts, like option pricing in a binomial tree, and then move on to some new ideas about how we can perfectly hedge away the underlying price risk through **delta hedging**.*

To begin with, let's see the type of underlying stock price distribution that we are going to be using in the following graph. Remember that, for simplicity, we are going to be using a risk-free rate $r = 0$, and that we will be working with (pricing) a call option with strike price $K = 90$.

Let's start importing some of the libraries that we will need to use down the road:

```
[1]: import numpy as np
```

1.1 1. Delta Hedging in the Binomial Tree

In the following section, you have the code for a general function to price a European call option using a binomial tree methodology. The function, as you can see, is able to compute the evolution of the underlying asset S_t , the price of the call option at each node of the tree, C_t , and also the Δ_t (number of shares in terms of exposure) in each node of the tree.

```
[2]: def call_option_delta(S_ini, K, T, r, u, d, N):  
    dt = T / N # Define time step  
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
```

```

C = np.zeros([N + 1, N + 1]) # call prices
S = np.zeros([N + 1, N + 1]) # underlying price
Delta = np.zeros([N, N]) # delta
for i in range(0, N + 1):
    C[N, i] = max(S_ini * (u ** (i)) * (d ** (N - i)) - K, 0)
    S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
for j in range(N - 1, -1, -1):
    for i in range(0, j + 1):
        C[j, i] = np.exp(-r * dt) * (p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i])
        S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
        Delta[j, i] = (C[j + 1, i + 1] - C[j + 1, i]) / (
            S[j + 1, i + 1] - S[j + 1, i]
        )
return C[0, 0], C, S, Delta

```

In order to understand properly the general function defined above, we are going to use the toy example from the course slides. First of all, we will map the example in the course slides with the parameters in the function “call_option_delta”.

- $N = 2$ (Two periods/time steps). Thus, we will have three times: $t = 0, t = 1, t = 2$
- Delta will be defined in three nodes: one node at $t = 0$, named Δ_0 , and two nodes (two potential outcomes) at $t = 1$: Δ_1^u and Δ_1^d .

If we run the “call_option_delta” function, we can print directly the underlying asset price, call price, and delta for all nodes as follows. As you can see, these match what you have in the slides:

```

[3]: price, call, S, delta = call_option_delta(100, 90, 15, 0, 1.2, 0.8, 15)
print("Underlying: \n", S)
print("Call Price: \n", call)
print("Delta: \n", delta)

```

Underlying:

```

[[ 100.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 80.      120.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 64.      96.      144.      0.      0.
   0.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 51.2      76.8      115.2      172.8      0.
   0.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]

```

[	40.96	61.44	92.16	138.24	207.36
	0.	0.	0.	0.	0.
	0.	0.	0.	0.	0.
	0.				
	]				
[	32.768	49.152	73.728	110.592	165.888
	248.832	0.	0.	0.	0.
	0.	0.	0.	0.	0.
	0.				
	]				
[	26.2144	39.3216	58.9824	88.4736	132.7104
	199.0656	298.5984	0.	0.	0.
	0.	0.	0.	0.	0.
	0.				
	]				
[	20.97152	31.45728	47.18592	70.77888	106.16832
	159.25248	238.87872	358.31808	0.	0.
	0.	0.	0.	0.	0.
	0.				
	]				
[	16.777216	25.165824	37.748736	56.623104	84.934656
	127.401984	191.102976	286.654464	429.981696	0.
	0.	0.	0.	0.	0.
	0.				
	]				
[	13.4217728	20.1326592	30.1989888	45.2984832	67.9477248
	101.9215872	152.8823808	229.3235712	343.9853568	515.9780352
	0.	0.	0.	0.	0.
	0.				
	]				
[	10.73741824	16.10612736	24.15919104	36.23878656	54.35817984
	81.53726976	122.30590464	183.45885696	275.18828544	412.78242816
	619.17364224	0.	0.	0.	0.
	0.				
	]				
[	8.58993459	12.88490189	19.32735283	28.99102925	43.48654387
	65.22981581	97.84472371	146.76708557	220.15062835	330.22594253
	495.33891379	743.00837069	0.	0.	0.
	0.				
	]				
[	6.87194767	10.30792151	15.46188227	23.1928234	34.7892351
	52.18385265	78.27577897	117.41366845	176.12050268	264.18075402
	396.27113103	594.40669655	891.61004483	0.	0.
	0.				
	]				
[	5.49755814	8.24633721	12.36950581	18.55425872	27.83138808
	41.74708212	62.62062318	93.93093476	140.89640215	211.34460322
	317.01690483	475.52535724	713.28803586	1069.93205379	0.
	0.				
	]				
[	4.39804651	6.59706977	9.89560465	14.84340697	22.26511046
	33.39766569	50.09649854	75.14474781	112.71712172	169.07568257
	253.61352386	380.42028579	570.63042869	855.94564303	1283.91846455
	0.				
	]				
[	3.51843721	5.27765581	7.91648372	11.87472558	17.81208837
	26.71813255	40.07719883	60.11579825	90.17369737	135.26054606
	202.89081909	304.33622863	456.50434295	684.75651443	1027.13477164
	1540.70215746]]				

Call Price:

```
[3.36896817e+01 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.98224454e+01 4.75569181e+01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.06139102e+01 2.90309806e+01 6.60828557e+01 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[4.98843591e+00 1.62393844e+01 4.18225767e+01 9.03431347e+01
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.94049089e+00 8.03638093e+00 2.44423879e+01 5.92027655e+01
1.21483504e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[5.59875646e-01 3.32110613e+00 1.27516557e+01 3.61331201e+01
8.22724110e+01 1.60694597e+02 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[9.14527782e-02 1.02829851e+00 5.61391375e+00 1.98893977e+01
5.23768424e+01 1.12167980e+02 2.09221214e+02 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[6.78505363e-04 1.82227051e-01 1.87436998e+00 9.35345752e+00
3.04253379e+01 7.43283469e+01 1.50007612e+02 2.68434815e+02
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 1.35701073e-03 3.63097091e-01 3.38564286e+00
1.53212722e+01 4.55294036e+01 1.03127290e+02 1.96887934e+02
3.39981696e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 2.71402145e-03 7.23480161e-01
6.04780556e+00 2.45947388e+01 6.64640685e+01 1.39790512e+02
2.53985357e+02 4.25978035e+02 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 5.42804291e-03
1.44153228e+00 1.06540788e+01 3.85353987e+01 9.43927383e+01
1.85188285e+02 3.22782428e+02 5.29173642e+02 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
1.08560858e-02 2.87220847e+00 1.84359492e+01 5.86348482e+01
1.30150628e+02 2.40225943e+02 4.05338914e+02 6.53008371e+02
```

```

0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 2.17121716e-02 5.72270477e+00 3.11491937e+01
8.61205027e+01 1.74180754e+02 3.06271131e+02 5.04406697e+02
8.01610045e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 4.34243432e-02 1.14019852e+01
5.08964021e+01 1.21344603e+02 2.27016905e+02 3.85525357e+02
6.23288036e+02 9.79932054e+02 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 8.68486865e-02
2.27171217e+01 7.90756826e+01 1.63613524e+02 2.90420286e+02
4.80630429e+02 7.65945643e+02 1.19391846e+03 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
1.73697373e-01 4.52605461e+01 1.12890819e+02 2.14336229e+02
3.66504343e+02 5.94756514e+02 9.37134772e+02 1.45070216e+03]]

```

Delta:

```

[[6.93361819e-01 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[5.75533450e-01 7.71914065e-01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[4.39490176e-01 6.66228966e-01 8.42370798e-01 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[2.97650881e-01 5.34049706e-01 7.54348473e-01 9.01052348e-01
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.68532134e-01 3.83730045e-01 6.34262813e-01 8.34405580e-01
9.45483527e-01 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[7.14756573e-02 2.33236452e-01 4.84059108e-01 7.34398616e-01
9.01076889e-01 9.75087952e-01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.73138185e-02 1.07583550e-01 3.17005053e-01 5.95428478e-01
8.27045375e-01 9.50431232e-01 9.91525766e-01 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.61768285e-04 2.87485187e-02 1.60140237e-01 4.21581597e-01
7.11326398e-01 9.04191359e-01 9.81257813e-01 9.98371068e-01

```

```

0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 2.69613808e-04 4.77344552e-02 2.35077425e-01
5.45917711e-01 8.21598856e-01 9.59253028e-01 9.95927670e-01
1.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 4.49356346e-04 7.92578545e-02
3.38957139e-01 6.83891426e-01 9.13403809e-01 9.89819174e-01
1.00000000e+00 1.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 7.48927244e-04
1.31597139e-01 4.77197139e-01 8.21687617e-01 9.74547936e-01
1.00000000e+00 1.00000000e+00 1.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
1.24821207e-03 2.18496424e-01 6.49664283e-01 9.36369841e-01
1.00000000e+00 1.00000000e+00 1.00000000e+00 1.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 2.08035345e-03 3.62773805e-01 8.40924602e-01
1.00000000e+00 1.00000000e+00 1.00000000e+00 1.00000000e+00
1.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 3.46725576e-03 6.02311504e-01
1.00000000e+00 1.00000000e+00 1.00000000e+00 1.00000000e+00
1.00000000e+00 1.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 5.77875960e-03
1.00000000e+00 1.00000000e+00 1.00000000e+00 1.00000000e+00
1.00000000e+00 1.00000000e+00 1.00000000e+00]]

```

According to the above results, we can conclude that, if we want to hedge the sale of this call option (remember, we are selling this as a bank), we will need to perform the following steps:

- At $t = 0$, we have to buy 0.675 shares (assuming that the underlying is a stock).
- At $t = 1$, we have to “hold” 0.1875 shares if we are “down” ($\Delta_1^d = 0.1875$) or 1 share if we are “up” ($\Delta_1^u = 1$).

To give you a complete idea of how the hedging system would work in practice, we will focus on the path, du .

Thus, at the first step, we follow the path “down”. That means that at $t = 0$, we buy 0.675 shares at \100 USD price per share = \67.5 USD total cost. At $t = 1$, we sell 0.4875 shares (0.675 - 0.1875 = 0.4875 shares) at \80 USD price to get \39 USD. Thus, we end up at $t = 1$ with a cost of \28.5 USD, but we still have 0.1875 shares.

Finally, if we end up following the path “ du ” (first down and then up), the final price of the underlying is \96 USD and the payoff of our call is \6 USD. Thus, we have at maturity, $+96 * 0.1875 - 28.5 =$

-10.5 USD. The price of the call in this path (“ du ”), as you can see in the above results, is equal to

$c_2^{du} = 6$ USD. This \\$6 USD is the payment that we need to fulfill with the buyer; thus, our total cost is $-10.5 - 6 = -\$16.5$ USD, which is the call price.

1.2 2. Generalization to Any N

It is easy, once the above example is properly understood, to generalize the binomial tree and the hedging strategy to any number of time steps. For instance, see the example below with $N=4$. All rights reserved WQU WorldQuant University QQQQ

```
[4]: price, call, S, delta = call_option_delta(100, 90, 10, 0, 1.2, 0.8, 10)
print("Underlying: \n", S)
print("Call Price: \n", call)
print("Delta: \n", delta)
```

Underlying:

```
[[100.      0.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 80.      120.      0.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 64.      96.      144.      0.      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 51.2      76.8      115.2      172.8      0.
   0.      0.      0.      0.      0.
   0.      ]
 [ 40.96      61.44      92.16      138.24      207.36
   0.      0.      0.      0.      0.
   0.      ]
 [ 32.768      49.152      73.728      110.592      165.888
  248.832      0.      0.      0.      0.
   0.      ]
 [ 26.2144      39.3216      58.9824      88.4736      132.7104
  199.0656      298.5984      0.      0.      0.
   0.      ]
 [ 20.97152      31.45728      47.18592      70.77888      106.16832
  159.25248      238.87872      358.31808      0.      0.
   0.      ]
 [ 16.777216      25.165824      37.748736      56.623104      84.934656
  127.401984      191.102976      286.654464      429.981696      0.
   0.      ]
 [ 13.4217728      20.1326592      30.1989888      45.2984832      67.9477248
  101.9215872      152.8823808      229.3235712      343.9853568      515.9780352
   0.      ]
 [ 10.73741824      16.10612736      24.15919104      36.23878656      54.35817984
   81.53726976      122.30590464      183.45885696      275.18828544      412.78242816
  619.17364224]]
```

Call Price:

```
[2.93845445e+01 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[1.57572106e+01 4.30118784e+01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[7.17743934e+00 2.43369819e+01 6.16867749e+01 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[2.49687648e+00 1.18580022e+01 3.68159616e+01 8.65575882e+01
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[5.04779760e-01 4.48897320e+00 1.92270312e+01 5.44048920e+01
1.18710284e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 1.00955952e+00 7.96838688e+00 3.04856755e+01
7.83241085e+01 1.59096460e+02 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 2.01911904e+00 1.39176547e+01
4.70536963e+01 1.09594521e+02 2.08598400e+02 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 4.03823808e+00
2.37970714e+01 7.03103213e+01 1.48878720e+02 2.68318080e+02
0.00000000e+00 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
8.07647616e+00 3.95176666e+01 1.01102976e+02 1.96654464e+02
3.39981696e+02 0.00000000e+00 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 1.61529523e+01 6.28823808e+01 1.39323571e+02
2.53985357e+02 4.25978035e+02 0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 3.23059046e+01 9.34588570e+01
1.85188285e+02 3.22782428e+02 5.29173642e+02]]
```

Delta:

```
[0.68136669 0. 0. 0. 0.
0. 0. 0. 0. ]
[0.53623571 0.77812069 0. 0. 0. 0.
0. 0. 0. 0. ]
[0.36566897 0.64994686 0.86356991 0. 0. 0.
0. 0. 0. 0. ]
[0.1945407 0.47975449 0.76340844 0.93034422 0. 0.
0. 0. 0. 0. ]
[0.06161862 0.28315541 0.61082055 0.8651337 0.9738179 0.
0. 0. 0. 0. ]
[0. 0.10269771 0.40346055 0.74906055 0.9425158 0.99468597
0. 0. 0. 0. ]
[0. 0. 0.17116284 0.55832568 0.87621712 0.98671492
```



```

1.      0.      0.      0.      ]
[0.      0.      0.      0.2852714  0.74036187  0.96678729
1.      1.      0.      0.      ]
[0.      0.      0.      0.      0.47545234  0.91696823
1.      1.      1.      0.      ]
[0.      0.      0.      0.      0.      0.79242056
1.      1.      1.      1.      ]]

```

The price of the call is equal to \ \$20.205 USD, which is quite different from the previous value of \ \$16.5 USD. You will notice that if you try with a higher N, the call price will increase monotonically. See the code snippet below:

1.2.1 2.1. Convergence

Next, let's check the power of the binomial pricing method. In the following code snippet, we calculate the price of the option for different N's. Execute the following code to see how the price of the option changes with N.

```

[5]: price_array = []
for N in [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000,
↪2500]:
    price, call, S, delta = call_option_delta(100, 90, 1, 0, 1.2, 0.8, N)
    price_array.append(price)
    print("With N = {:3d}, the price is {:.2f}".format(N, price))

```

```

With N = 1, the price is 15.00
With N = 10, the price is 29.38
With N = 100, the price is 70.32
With N = 200, the price is 85.40
With N = 300, the price is 92.33
With N = 400, the price is 95.84
With N = 500, the price is 97.70
With N = 600, the price is 98.71
With N = 700, the price is 99.27
With N = 800, the price is 99.58
With N = 900, the price is 99.76
With N = 1000, the price is 99.86
With N = 1500, the price is 99.99
With N = 2000, the price is 100.00
With N = 2500, the price is 100.00

```

In the next snippet, you can see this from a more visual perspective:

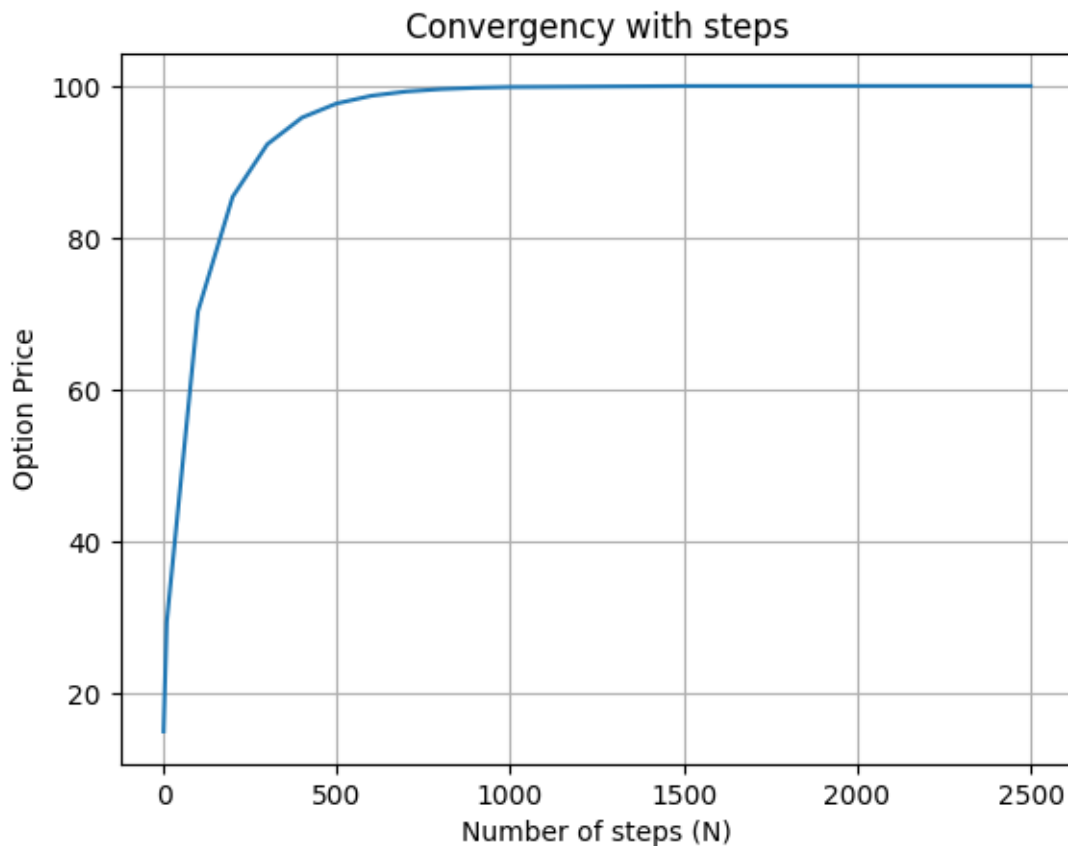
```

[6]: import matplotlib.pyplot as plt

N = [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000, 2500]
plt.plot(N, np.array(price_array))
plt.title("Convergency with steps")
plt.xlabel("Number of steps (N)")

```

```
plt.ylabel("Option Price")
plt.grid(True)
plt.show()
```



Of course, the maximum value of the call price must be the underlying price. The question is, **why are we getting this absurd behavior?**

1.3 3. Conclusion

We have gone over the basic steps to price options in a binomial model and explore how the prices we obtain change when we use a different number of steps in the model. However, the results from these experiments yielded somewhat inconsistent, or absurd, results.

In the next lesson, we will see that we need to adjust “ u ” and “ d ” with the number of steps (dt) in order to match the underlying asset’s volatility. Doing this, we will get rid of the previous problem.

See you in the next lesson!

References

- Hull, J. C. (2021). *Options futures and other derivatives*. 11th Edition. Pearson. Chapter 13.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

LESSON 4

MATCHING VOLATILITY AND RISK MEASURES

Derivative Pricing - Module 1



Outline

- ▶ Risk-neutral Valuation: Irrelevance of expected returns
- ▶ Real World vs. Risk-Neutral Measures: Girsanov's Theorem
- ▶ Intro to Calibration: Adjusting with Volatility when Increasing N in Binomial Tree

Risk-Neutral Valuation and Expected Stock Return

Throughout this module, we have worked with absence of arbitrage conditions and portfolio replication to obtain a fair price for an option (call or put).

One important feature that allows us to do that is risk-neutral valuation:

→ Derive risk-neutral probabilities and discount at risk-free rate.

$$S_0 = e^{-rT}[\mathbf{p}S_1^u + (1 - \mathbf{p})S_1^d] \rightarrow \text{with } \mathbf{p} \text{ risk-neutral prob}$$

$$\text{Lesson 1 example: } 100 = e^{-0.1 \times 1}[120\mathbf{p} + 80(1 - \mathbf{p})] \rightarrow \text{with } \mathbf{p} = 0.7629.$$

Notice that this implies expected stock return is completely irrelevant for valuation (which is exactly what we want, given how difficult it is to know this in practice).

However, the price should be exactly the same. For example, for $E(R) = \mu = 15\%$:

$$S_0 = e^{-\mu T}[\mathbf{p}^*S_1^u + (1 - \mathbf{p}^*)S_1^d] \rightarrow \text{with } \mathbf{p}^* \text{ real-world problem}$$

$$\text{Lesson 1 example: } 100 = e^{-0.15 \times 1}[120\mathbf{p}^* + 80(1 - \mathbf{p}^*)] \rightarrow \text{with } \mathbf{p}^* = 0.9046.$$

Real-World vs. Risk-Neutral Measures

If all information is available, we should be **indifferent between risk measures**:

- ▶ Real-world probabilities (p^*) are typically referred to as P-measure.
- ▶ Risk-neutral probabilities (p) are typically referred to as Q-measure.
- ▶ Be careful about matching risk-measure & discount rate.

How do we move between risk-neutral & real-world measures, and what implications does this have?

→ **Girsanov's Theorem!**

- ▶ To change the measure (move between risk-neutral & real worlds)
- ▶ When we change from one world to another, only two things change: discount rate & probability.
- ▶ **Volatility** of the underlying (e.g., stock) **does not change**.

⇒ We can avoid taking a stand on $E(R)$ of underlying, μ , with risk-neutral valuation.

⇒ What about volatility? → **Volatility will not change** when changing "world"

- ▶ *Match underlying stock volatility with u and d (and time-step, dt)*

Matching Volatility with u and d

In Lesson 3, we observed the absurd behavior of option prices when N increased... Why?

- We need to choose u and d to match the volatility of the underlying asset, σ !

So, u and d must be such that they match the variance of the underlying:

Variance $\rightarrow E(X^2) - [E(X)]^2 = \sigma^2 dt$, for dt equal time-step:

$$p(u-1)^2 + (1-p)(d-1)^2 - [p(u-1) + (1-p)(d-1)]^2 = \sigma^2 dt$$

Substituting for the value of p :

$$e^{rdt}(u+d) - ud - e^{2rdt} = \sigma^2 dt$$

Using series expansion of e^x and ignoring power terms dt^2 and higher ($\rightarrow 0$):

$$u = e^{\sigma\sqrt{dt}} \text{ and } d = e^{-\sigma\sqrt{dt}}$$

$\Rightarrow$ Remember, these expressions are **the same** regardless of risk-neutral or real worlds!

$$p = \frac{e^{rdt} - d}{u - d} \text{ and } p^* = \frac{e^{\mu dt} - d}{u - d}$$

Summary of Lesson 4

In Lesson 4, we covered the following topics:

- ▶ Differences between risk-neutral vs. real-world probability measures
- ▶ Importance of matching discount rate & probability measure
- ▶ Volatility does not change with world $\rightarrow$ Need to adjust u and d

You can find more information on the derivation of binomial model formulas for u and d (and more) in the original paper:

- ▶ Cox, John C., Stephen A. Ross, and Mark Rubinstein. "Option Pricing: A Simplified Approach." *Journal of Financial Economics*, vol. 7, 1979, 229–263.

$\Rightarrow$ **TO-DO NEXT:** In the Jupyter Notebook accompanying this lesson, you will find code developing the adjustments of u and d to underlying stock volatility. This will serve as a first introduction to the concept of calibration.

$\Rightarrow$ In the next module, we will revisit and reinforce these concepts for other types of derivatives, such as American options.

Lesson 4 - Matching Volatility and Risk-measures

February 6, 2026

0.1 DERIVATIVE PRICING

MODULE 1 | LESSON 4

1 MATCHING VOLATILITY AND RISK-MEASURES

Reading Time	45 minutes
Prior Knowledge	Risk-neutral probability, Volatility, Time-step
Keywords	Model calibration, Matching volatility

In the final notebook for Module 1, we will put all the previous concepts together to perform a complete valuation of a call option adjusting the binomial tree inputs u and d for the volatility of the underlying asset.

*This adjusting step is the foundation of **model calibration**, which will be one of the keystones of the course and of derivative pricing in general.*

Let's start importing some of the libraries that we will need to use down the road:

```
[1]: import numpy as np
```

1.1 1. Adjusting u and d for Underlying Volatility

Let's start by adjusting the inputs u and d to match underlying volatility. As we have covered in the notes/videos for the lesson, we know that adjustment will lead to:

$$u = e^{\sigma\sqrt{dt}} \text{ and } d = e^{-\sigma\sqrt{dt}}$$

Now, assume that we know the underlying stock volatility for the next year (e.g., $\sigma = 30\%$). (*For now, let's take this number as given.*)

We can modify our previous functions to incorporate the volatility adjustment to u and d : All rights reserved WQU WorldQuant University QQQQ

```
[2]: def call_option_delta(S_ini, K, T, r, sigma, N):  
    dt = T / N # Define time step  
    u = np.exp(sigma * np.sqrt(dt)) # Define u  
    d = np.exp(-sigma * np.sqrt(dt)) # Define d
```

```

p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
C = np.zeros([N + 1, N + 1]) # call prices
S = np.zeros([N + 1, N + 1]) # underlying price
Delta = np.zeros([N, N]) # delta
for i in range(0, N + 1):
    C[N, i] = max(S_ini * (u ** (i)) * (d ** (N - i)) - K, 0)
    S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
for j in range(N - 1, -1, -1):
    for i in range(0, j + 1):
        C[j, i] = np.exp(-r * dt) * (p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i])
        S[j, i] = S_ini * (u ** (i)) * (d ** (j - i))
        Delta[j, i] = (C[j + 1, i + 1] - C[j + 1, i]) / (
            S[j + 1, i + 1] - S[j + 1, i]
        )
return C[0, 0], C, S, Delta

```

1.2 2. Convergence

Let's see what the call option ($K = 90$, $r = 0\%$, $T = 1$, $\sigma = 0.3$) price is for different N 's:

```

[3]: price_array = []
for N in [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000, 2500]:
    call_price, C, S, delta = call_option_delta(100, 90, 1, 0, 0.3, N)
    price_array.append(call_price)
    print("With N = {:3d}, the price is {:.2f}".format(N, call_price))

```

```

With N = 1, the price is 19.14
With N = 10, the price is 17.26
With N = 100, the price is 17.02
With N = 200, the price is 17.03
With N = 300, the price is 17.01
With N = 400, the price is 17.02
With N = 500, the price is 17.01
With N = 600, the price is 17.02
With N = 700, the price is 17.02
With N = 800, the price is 17.01
With N = 900, the price is 17.01
With N = 1000, the price is 17.02
With N = 1500, the price is 17.01
With N = 2000, the price is 17.01
With N = 2500, the price is 17.01

```

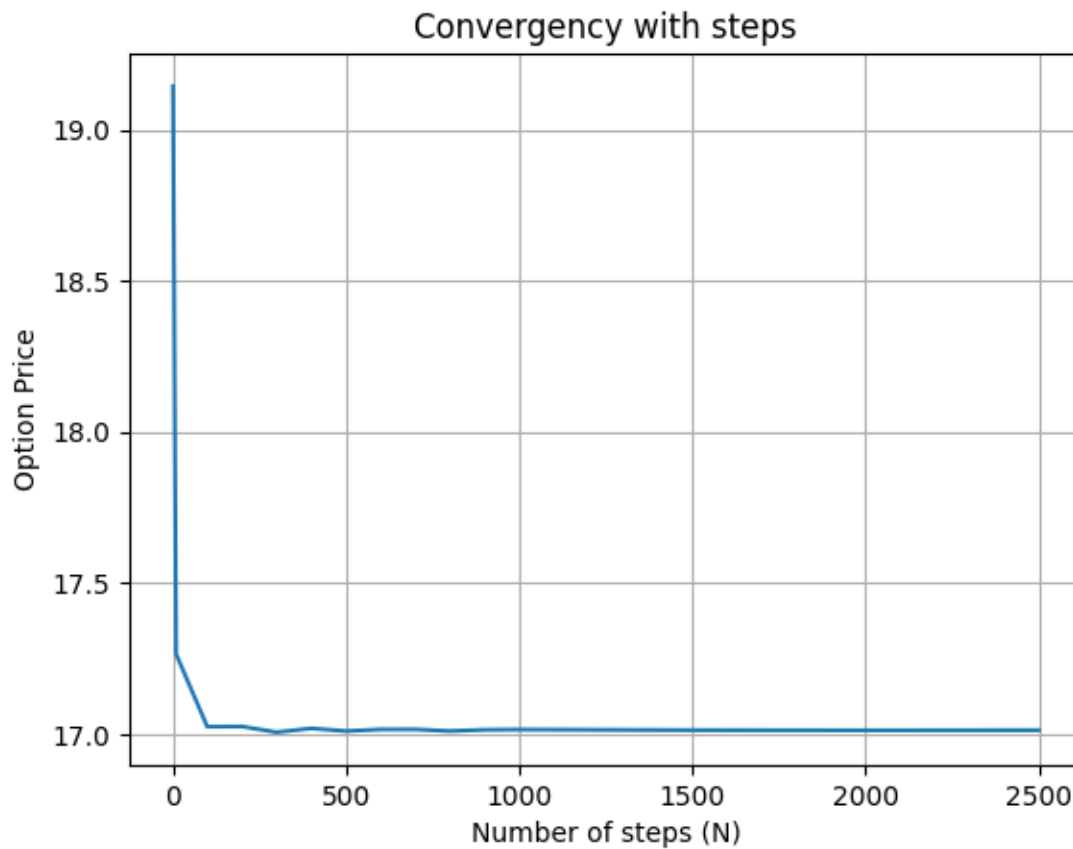
```

[4]: import matplotlib.pyplot as plt

N = [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000, 2500]
plt.plot(N, np.array(price_array))

```

```
plt.title("Convergency with steps")
plt.xlabel("Number of steps (N)")
plt.ylabel("Option Price")
plt.grid(True)
plt.show()
```



As we can see in the graph above, the price of the call option now clearly converges to a price of $c = 17.01$.

1.3 3. How Do We Obtain σ ?

Obtaining the volatility of the underlying asset in order to calibrate the model is one of the most challenging tasks that we will face.

For now, we have taken this input as a given. However, in reality, we would need to estimate this somehow. Can you think of how to do it?

- Historical asset volatility (e.g., last year) ? → But this relies on the past = No good
- Implied asset volatility ? → implied by what? → Exchange-traded option prices

We will do a deep analysis of the latter point during the course. For now, just think about the

following idea:

If we have an exchange-traded option (for which the **price is set by supply and demand**) and a **pricing equation** that depends on parameters (including σ), we can resolve this equation and **extract the volatility implied by that price**.

We will come back to this concept in the future. For now, you can explore this by looking at the Options Chain for Apple (AAPL) [in Yahoo! Finance](#).

1.4 4. Conclusion

Throughout this lesson, we have worked on a simple model calibration, something that will become very relevant when dealing with more complex models. We will be constantly revisiting model calibration throughout the course, so make sure you grasp the concept fully.

In the next module, we will revisit most of these ideas for the case of American options.

See you in Module 2!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.